```
Event STATE_TRANSITION published: CircuitBreaker 'orderCircuitBreaker'
  changed state from CLOSED to OPEN
```

The 21st request will not even try contacting Order Service: the circuit is open, so it cannot go through. A `NOT_PERMITTED` event is registered to signal why the request failed:

```
Event NOT_PERMITTED published: CircuitBreaker 'orderCircuitBreaker'
  recorded a call which was not permitted.
```

NOTE Monitoring the status of circuit breakers in production is a critical task. In chapter 13, I'll show you how to export that information as Prometheus metrics that you can visualize in a Grafana dashboard instead of checking the logs. In the meantime, for a more visual explanation, feel free to watch my "Spring Cloud Gateway: Resilience, Security, and Observability" session on circuit breakers at Spring I/O, 2022 (http://mng.bz/z55A).

Now let's see what happens when we call a `GET` endpoint for which both retries and fallback have been configured. Before proceeding, rerun the application so you can start with a clear circuit breaker state (`./gradlew bootRun`). Then run the following command:

```
$ ab -n 21 -c 1 -m GET http://localhost:9000/books
```

If you check the application logs, you'll see how the circuit breaker behaves precisely like before: 20 allowed requests (closed circuit), followed by a non-permitted request (open circuit). However, the result of the previous command shows 21 requests completed with no errors:

```
Complete requests: 21
Failed requests: 0
```

This time, all requests have been forwarded to the fallback endpoint, so the client didn't experience any errors.